Module 4

CSS Fundamentals

Agenda

1. Introduction to CSS
2. Applying CSS
3. CSS Selectors, Classes and IDs
4. Colors, Backgrounds ,Typography and Fonts
5. CSS Units
6. CSS Specificity, Cascade and Inheritance

© Copyright Intellipaat. All rights reserved.

Introduction to CSS

© Copyright Intellipaat. All rights reserved.

What is CSS?

CSS (**Cascading Style Sheets**) is a **stylesheet language used to control the presentation and visual appearance of HTML elements**.

HTML provides the elements → **CSS** styles those elements.



CSS can be used to control several aspects of a webpage:

- **Colors** — text, backgrounds, borders
- **Typography** — fonts, sizes, weights, line spacing
- **Spacing** — margins and padding
- **Dimensions** — width and height
- **Appearance** — backgrounds, borders, etc.
- **Layout** — how elements are arranged on the page
- **Positioning** — where elements appear
- **Responsive presentation** — how the page adapts to different screen sizes

© Copyright Intellipaat. All rights reserved.

What is CSS?

CSS doesn't create the HTML elements themselves. Instead, it **applies styling rules to existing HTML elements**.

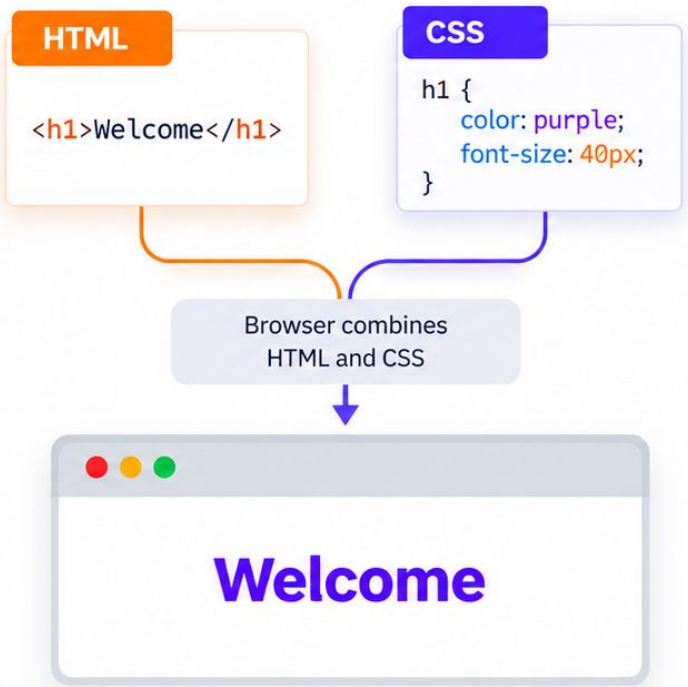
1. For example, if HTML contains:

```html
<h1>Welcome</h1>
```

2. CSS can tell the browser:

```css
h1 {
    color: purple;
    font-size: 40px;
}
```

3. The browser combines the HTML and CSS and **displays the styled result.**



© Copyright Intellipaat. All rights reserved.

How CSS Works

The fundamental idea is that CSS consists of rules that tell the browser, **which HTML elements should be styled**, **what property should change**, and **what value should that property have.**

When a browser loads a webpage, CSS goes through a process roughly like this:

1. The browser reads the HTML
2. The browser reads the CSS
3. CSS matches selectors to HTML elements
4. The browser resolves conflicts
5. The browser calculates the final appearance
6. The browser renders the page



© Copyright Intellipaat. All rights reserved.

CSS Rule Example

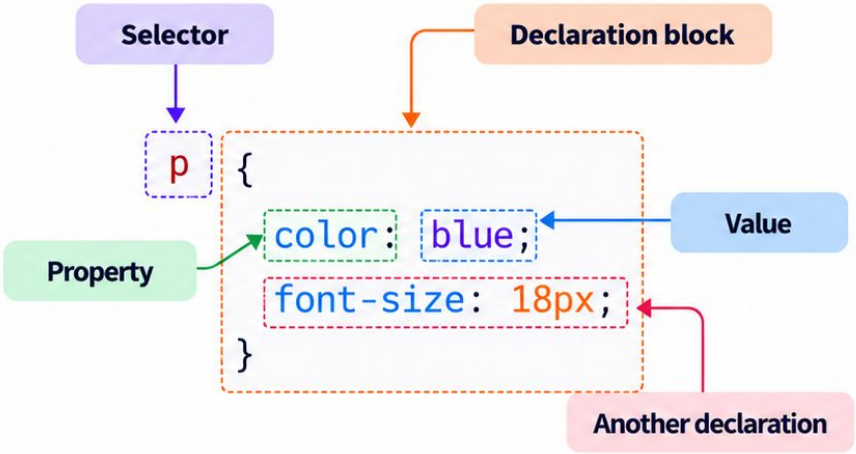
A CSS rule generally consists of: **Selector + Declaration Block**

For example:

```
p {
   color: blue;
   font-size: 18px;
}
```

Here:

- `p` → **Selector**
- `{ ... }` → **Declaration block**
- `color: blue;` → **Declaration**
- `color` → **Property**
- `blue` → **Value**
- `font-size: 18px;` → Another declaration



© Copyright Intellipaat. All rights reserved.

Hands-On 1 — Style a Simple Welcome Page

Create a simple HTML page containing:

- H1 — A heading: **Welcome to CSS**
- A paragraph: **Learn CSS step by step.**

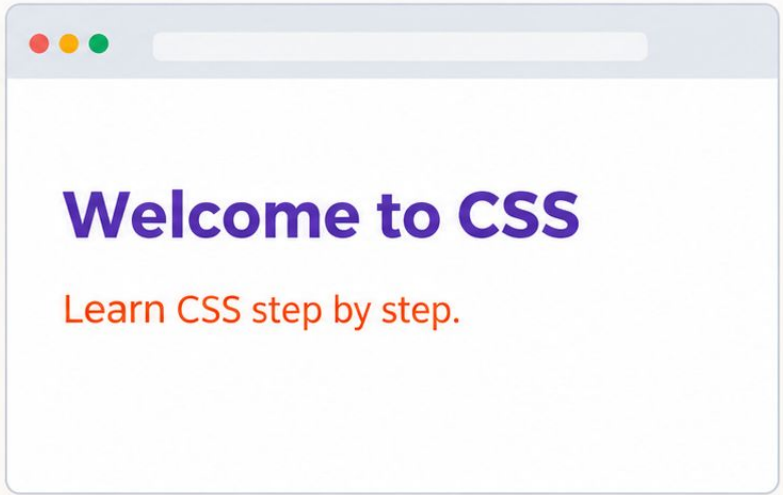
Then use CSS to change the **presentation and visual appearance** of the elements.

Your Task

1. Create the HTML elements.
2. Write a CSS rule for the heading.
3. Change the heading's text color.
4. Change the heading's font size.
5. Change the paragraph's text color.

Expected Result

A basic webpage where the heading and paragraph appear with the styles you defined.



© Copyright Intellipaat. All rights reserved.

Applying CSS

© Copyright Intellipaat. All rights reserved.

Inline CSS

Inline CSS is CSS written **directly inside an HTML element** using the style attribute.

1 The `style` attribute is added to the HTML element you want to style:

HTML

```
<h1 style="color: purple; font-size: 32px;">
  Welcome
</h1>
```

2 **Here:**

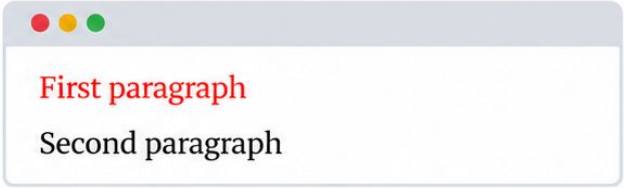
- `style` is an HTML attribute.
- The CSS declarations are written inside the attribute.
- The styles apply **only to that particular HTML element.**

3 Inline CSS has a **very narrow scope** because the styling is attached directly to one element.

For example:

HTML

```
<p style="color: red;">First paragraph</p>
<p>Second paragraph</p>
```



The first paragraph becomes red, while the second paragraph is unaffected.

© Copyright Intellipaat. All rights reserved.

Internal CSS

Internal CSS is CSS written inside a <style> element within the HTML document, usually inside the <head> section.

1 A <style> element is placed in the HTML document:

HTML

```
<head>
    <style>
        h1 {
            color: purple;
        }

        p {
            font-size: 18px;
        }
    </style>
</head>
```



2 The CSS rules inside <style> can then apply to matching elements throughout that HTML document.

© Copyright Intellipaat. All rights reserved.

Internal CSS

1 Internal CSS keeps the CSS rules **separate from the individual HTML elements**, while still keeping everything inside the **same HTML file**.

For example, one rule:

CSS

```
p {
   color: blue;
}
```

can style multiple `<p>` elements in that page.

2 The styles are generally available to the elements in **that particular HTML document**.

If you have:

- index.html
- about.html
- contact.html

and the same CSS is written internally in `index.html`, it does **not automatically style** the elements in `about.html` or `contact.html`.

© Copyright Intellipaat. All rights reserved.

External CSS

External CSS is CSS written in a **separate .css file** and then connected to an HTML document.

This separates the web page's **HTML structure** from its **CSS styling**.

Suppose you have:

- index.html
- style.css

index.html

```
<!DOCTYPE html>
<html>
   <head>
      <title>My Page</title>
      <link rel="stylesheet"
            href="style.css">
   </head>
   <body>
      <h1>Welcome</h1>
      <p>This is a paragraph.</p>
   </body>
</html>
```

Linked using <link> tag

style.css

```
h1 {
   color: purple;
}

p {
   font-size: 18px;
}
```

The CSS rules in **style.css** are applied to the matching elements in **index.html**.

© Copyright Intellipaat. All rights reserved.

External CSS

1 The HTML document connects to that stylesheet using the **<link>** element:

index.html

```
<head>
   <link rel="stylesheet" href="style.css">
</head>
```

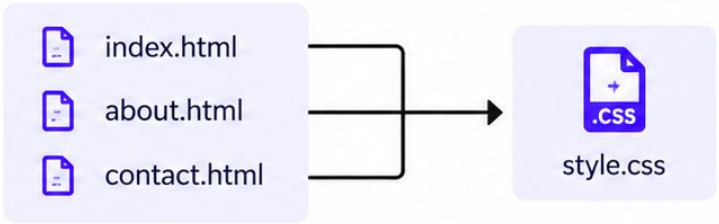
2 The browser then loads the CSS file and applies its rules to the matching HTML elements.

This beeps the **HTML structure** and **CSS styling** in separate files.

3 The CSS is **completely separate** from the HTML file.

4 This means the same stylesheet can be connected to **multiple HTML pages.**

For example:



All three pages can use the same CSS file.

© Copyright Intellipaat. All rights reserved.

Inline vs. Internal vs. External CSS

01 Inline CSS

CSS is written **directly on the HTML element** using the `style` attribute.

HTML

```
<p style="color: red;">Hello</p>
```

- Written inside the HTML element
- Applies to that specific element
- Useful for a one-off style
- Difficult to maintain when used extensively

02 Internal CSS

CSS is written inside a `<style>` element in the HTML document.

HTML

```
<style>
  p {
    color: red;
  }
</style>
```

- Written inside the HTML document
- Can style multiple elements on that page
- Useful when styling is specific to one page
- Cannot be directly reused by other pages

03 External CSS

CSS is written in a separate `.css` file.

style.css

```
p {
  color: red;
}
```

and connected to HTML:

HTML

```
<link rel="stylesheet" href="style.css">
```

- Written in a separate CSS file
- Can style multiple elements
- Can be shared across multiple HTML pages
- Easier to maintain and reuse

© Copyright Intellipaat. All rights reserved.

Hands-On 2 — Style the Same Page Using Three CSS Methods

Create a page containing:

- One heading
- Two paragraphs

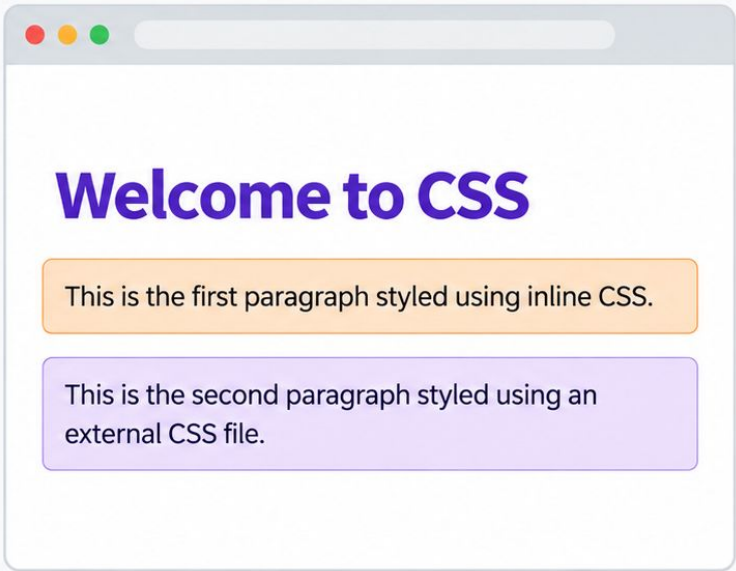
Apply CSS using all three methods:

1. **Inline CSS** — style the first paragraph.
2. **Internal CSS** — style the heading.
3. **External CSS** — style the second paragraph.

Your Task

1. Create a `style.css` file.
2. Connect it to your HTML using `<link>`.
3. Use the `<style>` element for the heading.
4. Use the `style` attribute for the first paragraph.
5. Use the external stylesheet for the second paragraph.

Visual Preview



Expected Result: All three elements are styled using their respective CSS methods.

© Copyright Intellipaat. All rights reserved.

CSS Selectors, Classes & IDs

© Copyright Intellipaat. All rights reserved.

CSS Selectors

A **CSS selector** is the part of a CSS rule that tells the browser **which HTML element or elements the rule should target**.

1 Why Selectors?

A webpage can contain many HTML elements. CSS needs a way to target specific elements rather than applying every style to everything.

For example, you might want:

- All paragraphs to have a certain font size
- Certain elements to have a particular color
- One specific element to have a unique style

Selectors provide that targeting mechanism.

2 What Selectors Do (and Don't Do)

A selector **doesn't change anything by itself**.
It identifies the elements to which the CSS declarations should be applied.

3 Example

CSS

```
p {
    color: blue;
}
```

The selector identifies the `<p>` elements, while the declaration `color: blue` defines **what should happen to them**.

© Copyright Intellipaat. All rights reserved.

Classes and IDs

Classes and IDs are HTML attributes used to **identify elements** so that CSS can style specific elements.

1 CSS Classes

A **class** is an HTML attribute used to assign an element to a particular group or category.

HTML

```
<p class="highlight">Important text</p>
<p class="highlight">Another important text</p>
```

2 CSS Syntax

CSS targets a class using a **period (.)**:

CSS

```
.highlight {
  color: red;
}
```

3 Output

Result

Important text

Another important text

The **same class** can be assigned to **multiple elements.**

This makes classes useful when several elements should **share the same styling.**

© Copyright Intellipaat. All rights reserved.

Classes and IDs

IDs

An **ID** is an HTML attribute used to **uniquely identify an element** within a page.

HTML Example:

HTML

```html
<h1 id="main-title">Welcome</h1>
```

CSS Example:

CSS

```css
#main-title {
  color: purple;
}
```

CSS targets an ID using a **hash (#)**.

An ID is intended to identify **one unique element** on a page.

© Copyright Intellipaat. All rights reserved.

Class vs. ID

The fundamental difference is **reusability:**

Class

- Can be used on **multiple elements**
- Targeted with **. (period)**
- Used for **grouping** elements

Example (Class)

HTML

```
<p class="card">Card 1</p>
<p class="card">Card 2</p>
<p class="card">Card 3</p>
```

One `.card` rule can style all three elements.

ID

- Intended for **one unique element**
- Targeted with **# (hash)**
- Used for **identifying** a specific element

Example (ID)

HTML

```
<h1 id="page-title">My Website</h1>
```

`#page-title` identifies that particular element.

© Copyright Intellipaat. All rights reserved.

Hands-On 3 — Style Elements Using Selectors, Classes and IDs

IntelliPaat

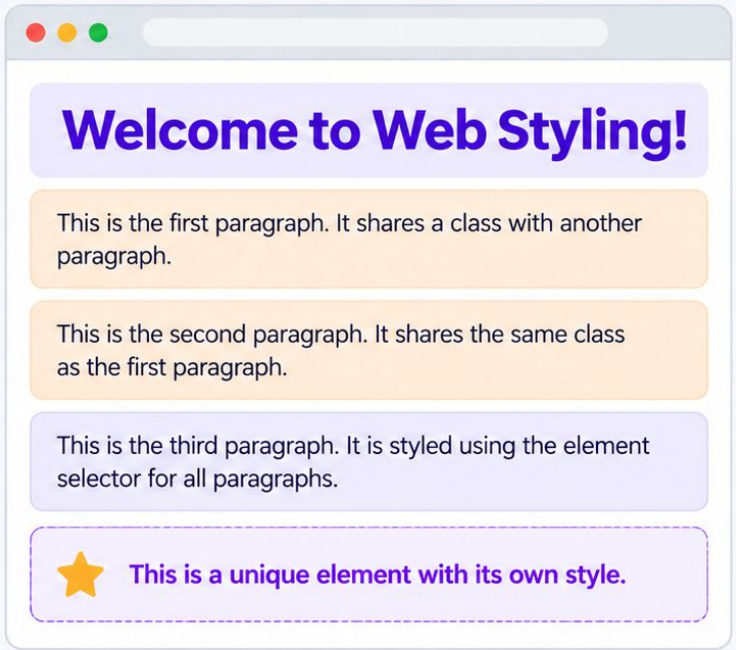
Create a webpage containing:

- One `<h1>` heading
- Three `<p>` elements
- One unique element that needs its own style

Your Task

1. Use an **element selector** to style all paragraphs.
2. Give two paragraphs the same **class** and apply a a shared style.
3. Give the heading a unique **ID** and style it using the **ID** selector.
4. Observe how the class can be reused while the ID ! identifies one specific element.

Visual Preview (Example Layout)



Expected Result: Common elements share styles through the class/element selectors, while the uniquely identified element receives its specific style.

© Copyright Intellipaat. All rights reserved.

Colors, Backgrounds, Typography & Fonts

© Copyright Intellipaat. All rights reserved.

Colors and Backgrounds

CSS provides properties that allow us to control the **colors and backgrounds** of HTML elements.

01 Text and Foreground Color

The `color` property controls the color of an **element's text.**

```css
p {
   color: blue;
}
```

The value can be specified in different ways, such as:

1. **Color names** — `red, blue, black`
2. **HEX values** — `#ff0000`
3. **RGB values** — `rgb(255, 0, 0)`

The `color` property controls the **text color,** not the background.

02 Background Color

The `background-color` property controls the **background color of an element.**

```css
div {
   background-color: lightblue;
}
```

This applies a solid color behind the element's content.

Basic Distinction:

`color`	→ Text color
`background-color`	→ Background color

© Copyright Intellipaat. All rights reserved.

Colors and Backgrounds

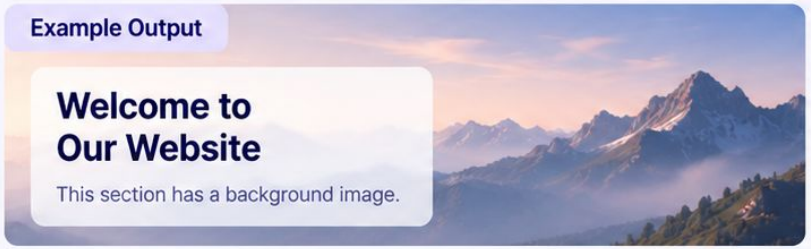
Background Image

CSS can also place an image in the background using the `background-image` .

CSS

```css
div {
   background-image: url("image.jpg");
}
```

Example Output



Unlike an `<img>` element, a background image is treated as part of the **element's visual styling**, rather than as content within the HTML structure.

Controlling the Background

CSS provides additional background properties.

CSS

```css
.hero {
  background-image: url("image.jpg");
  background-size: cover;
  background-position: center;
  background-repeat: no-repeat;
}
```

These properties control:

1. `background-size` → How large the image should be.
2. `background-position` → Where the image should appear.
3. `background-repeat` → Whether the image should repeat.

© Copyright Intellipaat. All rights reserved.

Example

IntelliPaat

HTML

```html
<div class="card">
    <h2>Welcome</h2>
    <p>Learn CSS step by step.</p>
</div>
```

CSS

```css
.card {
    color: white;
    background-color: purple;
    padding: 20px;
}
```

Result



The result is a card with a **purple background, white text**, and **spacing around its content**.

Key Idea

color	→	changes text color
background-color	→	changes background color
background-image	→	adds a background image

© Copyright Intellipaat. All rights reserved.

Typography and Fonts

Typography is the way text is **styled, arranged, and displayed** on a webpage. CSS provides properties to control how text looks and how easy it is to read.

Aa Font Family

The `font-family` property specifies **which font** should be used.

CSS

```css
body {
  font-family: Arial, sans-serif;
}
```

You can provide multiple fonts as fallbacks:

CSS

```css
body {
  font-family: "Arial", "Helvetica", sans-serif;
}
```

If the first font is unavailable, the browser tries the next one.

T Font Size

The `font-size` property controls **how large the text appears**.

CSS

```css
h1 {
  font-size: 40px;
}
```

CSS

```css
p {
  font-size: 18px;
}
```

Result

Welcome to Our Website

Learn CSS step by step.

© Copyright Intellipaat. All rights reserved.

Typography and Fonts

The **em** unit is a relative CSS unit that is based on the **font size of the parent element**.
For example, if the parent font size is **16px**, then **2em** equals **32px**.

Font Weight

The `font-weight` property controls **how thick or bold the text is.**

CSS

```css
h2 {
  font-weight: bold;
}
```

CSS

```css
p {
  font-weight: 400;
}
```

Common values include:

1. `400` → Normal
2. `500` → **Medium**
3. `600` → **Semi-bold**
4. `700` → **Bold**

Font Style

The `font-style` property controls whether text is **normal or italic.**

CSS

```css
em {
  font-style: italic;
}
```

Common values:

1. `normal` → Normal text
2. `italic` → *Italic text*
3. `oblique` → *Oblique text*

© Copyright Intellipaat. All rights reserved.

Typography and Fonts

Line Height

The `line-height` property controls the **vertical space between lines of text.**

CSS

```
p {
   line-height: 1.6;
}
```

Example

A suitable line height makes paragraphs easier to read. It gives the text more breathing room and improves readability.



A suitable line height makes paragraphs **easier to read.**

Text Alignment

The `text-align` property controls the **horizontal alignment of text.**

CSS

```
h1 {
   text-align: center;
}
```

Common Values

`text-align: left;`



Left Align

`text-align: center;`



Center Align

`text-align: right;`



Right Align

© Copyright Intellipaat. All rights reserved.

Typography and Fonts

A↔ Letter Spacing

The `letter-spacing` property controls the **space between individual letters**.

CSS

```css
h1 {
  letter-spacing: 2px;
}
```

Example

Normal Text	Learn CSS Step by Step
With Letter Spacing	L e a r n C S S S t e p b y S t e p

Letter spacing is useful for headings or design purposes to make text look more **clear and readable**.

T Text Decoration

The `text-decoration` property adds or removes decorations such as **underlines**.

CSS

```css
a {
  text-decoration: none;
}
```

For example:

CSS

```css
h2 {
  text-decoration: underline;
}
```

Example

This is Underlined Text

`text-decoration: underline;`

This is Normal Text

`text-decoration: none;`

© Copyright Intellipaat. All rights reserved.

Examples

HTML:

```html
<h1>Learn Web Development</h1>

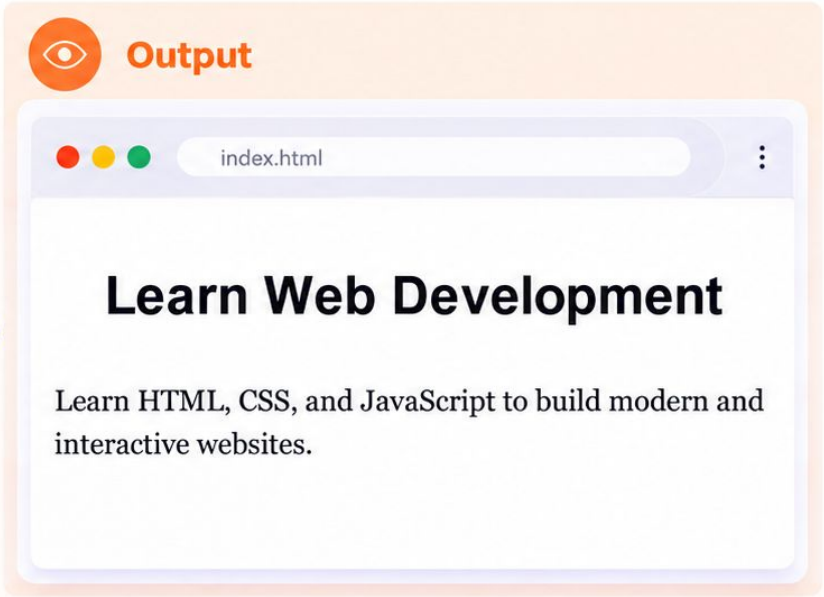
<p>
  Learn HTML, CSS, and JavaScript to build modern and interactive websites.
</p>
```

CSS:

```css
h1 {
  font-family: Arial, sans-serif;
  font-size: 40px;
  font-weight: 700;
  font-style: normal;
  text-align: center;
  letter-spacing: 1px;
}

p {
  font-family: Georgia, serif;
  font-size: 18px;
  font-weight: 400;
  line-height: 1.6;
  text-align: left;
}
```

Output



© Copyright Intellipaat. All rights reserved.

Hands-On 4 — Design a Styled Content Card

Create a `<div>` containing:

- A heading: **Learn Web Development**
- A paragraph: **Learn HTML, CSS, and JavaScript.**

Your Task

Use CSS to:

1. Set the **text color.**
2. Set a **background color** for the card.
3. Add a **background image.**
4. Set the **font family.**
5. Change the **font size.**
6. Change the **font weight.**
7. Change the **font style.**
8. Set the **line height.**
9. Align the text.
10. Adjust **letter spacing.**
11. Add or remove **text decoration** where appropriate.

Visual Preview (Example Output)



Expected Result: A visually styled content card demonstrating the typography and background properties learned in the topic.

© Copyright Intellipaat. All rights reserved.

CSS Units

© Copyright Intellipaat. All rights reserved.

CSS Units

CSS properties often require **numeric values** to define things such as size, spacing, width, height, and position. CSS units tell the browser **what that number represents**.

Different units behave differently. Some represent a **fixed size**, while others depend on another value, such as the size of a parent element or the browser's viewport.

This is especially important when creating layouts that need to work across different screen sizes.

Key Takeaway

CSS units help the browser understand what a number represents, ensuring consistent and responsive designs.

Two Broad Categories

The units in this module can be understood as:

Absolute Unit

Represents a **fixed size**.

`px`

`px` represents a specific CSS pixel measurement.

Relative Units

Depend on another value, such as the size of a parent element or the browser's viewport.

- `%`
- `em`
- `rem`
- `vh`
- `vw`

The important distinction is that **relative units depend on something else**, while `px` represents a specific CSS pixel measurement.

© Copyright Intellipaat. All rights reserved.

Absolute and Relative Units

Absolute Units

An absolute unit represents a **specific measurement** rather than being calculated from another element or the viewport.

For this module, the main absolute unit is:

`px` — **pixel**

CSS Example

```css
.box {
  width: 300px;
}
```



The value is specified directly as `300px`.

However, it's worth teaching students that a CSS pixel is **not necessarily one physical screen pixel.** CSS pixels are logical units used by the browser.

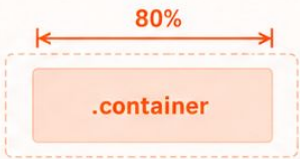
Relative Units

Relative units define a value **relative to something else.** The reference depends on the particular unit.

For example:

CSS Example

```css
.container {
  width: 80%;
}
```



The `80%` is calculated relative to the relevant containing size rather than representing one fixed measurement.

The relative units in this module are:

- `%` — relative to a relevant parent/container value
- `em` — relative to the font size of the relevant element/context
- `rem` — relative to the root element's font size
- `vh` — relative to the viewport height
- `vw` — relative to the viewport width

© Copyright Intellipaat. All rights reserved.

Types of Absolute and Relative Units

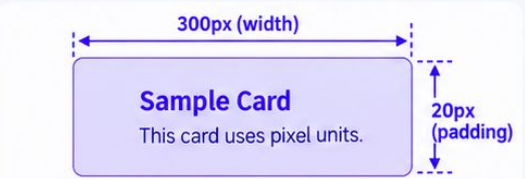
1 px — Pixels

`px` is a **fixed-size** unit commonly used for precise measurements.

```css
.card {
  width: 300px;
  padding: 20px;
  font-size: 18px;
}
```

Here:

- Width = `300px`
- Padding = `20px`
- Font size = `18px`



Think: `px` → **fixed measurement**

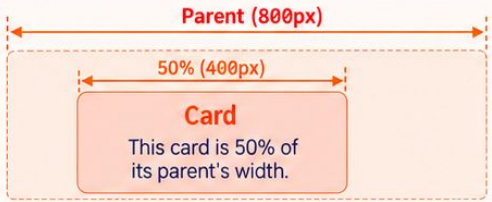
2 % — Percentage

`%` defines a size relative to the **parent element**.

```css
.container {
  width: 800px;
}

.card {
  width: 50%;
}
```

The card will be **50%** of its parent's width, so in this example it will be `400px`.



Think: `%` → **relative to the parent**

3 rem — Root EM

`rem` is relative to the **root** `(html)` font size.

If the root font size is `16px`:

```css
html {
  font-size: 16px;
}

h1 {
  font-size: 2rem;
}
```

Then:

`2rem = 2 × 16px = 32px`

`rem` is especially useful for creating **consistent and scalable typography and spacing.**

Think: `rem` → **relative to the root font size**

© Copyright Intellipaat. All rights reserved.

Types of Absolute and Relative Units

4 em — Relative to Font Size

`em` is relative to the **font size of the relevant element/context.**

CSS

```css
.card {
  font-size: 20px;
}

.card h2 {
  font-size: 1.5em;
}
```

The `h2` will be:

1.5 × 20px = 30px

Card Text	Heading (h2)
20px	30px (1.5em)

`em` is useful when you want sizes to **scale relative to surrounding text.**

5 vh — Viewport Height

`vh` means **viewport height.**

CSS

```css
.hero {
  height: 100vh;
}
```

`100vh` means the element will be as tall as the browser's viewport.

For example, if the viewport height is `800px`:

100vh = 800px

Similarly:

```css
.hero {
  height: 50vh;
}
```

means the element will be **50% of the viewport height.**





6 vw — Viewport Width

`vw` means **viewport width.**

CSS

```css
.hero {
  width: 100vw;
}
```

`100vw` represents the viewport's width.

For example, if the viewport is `1200px` wide:

100vw = 1200px

Similarly:

```css
h1 {
  font-size: 5vw;
}
```

makes the font size scale with the viewport width.



© Copyright Intellipaat. All rights reserved.

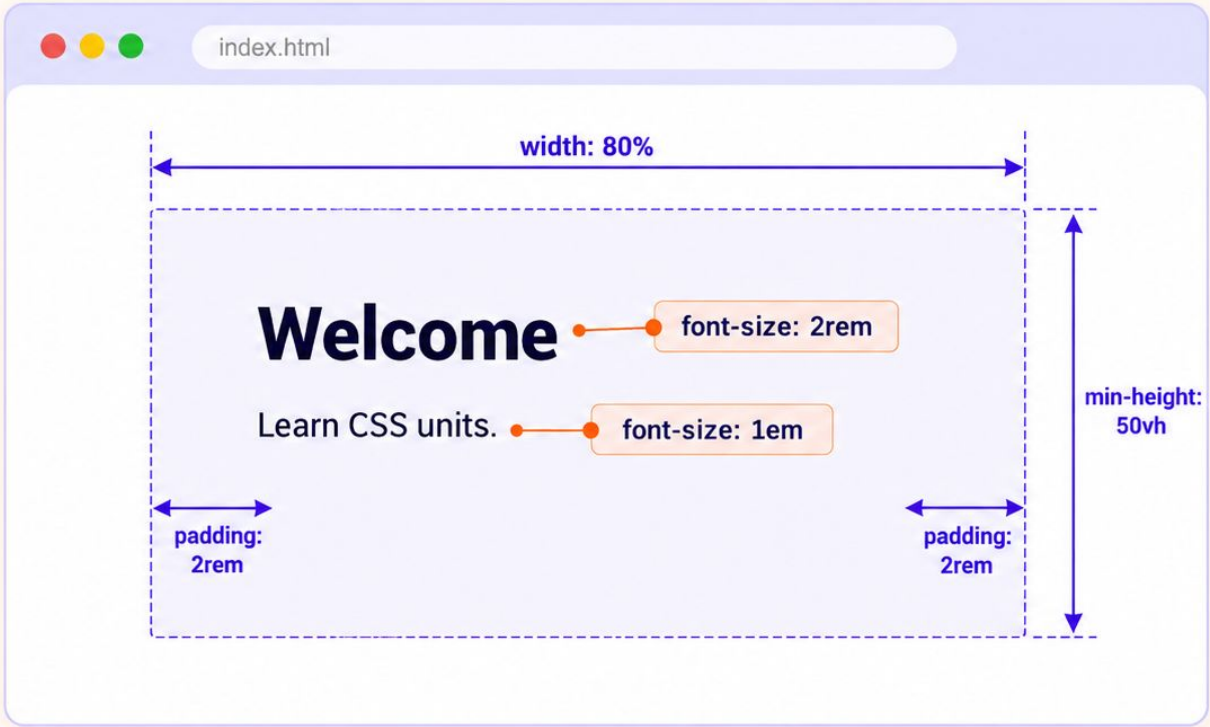
Example

HTML

```html
<div class="container">
  <h1>Welcome</h1>
  <p>Learn CSS units.</p>
</div>
```

CSS

```css
html {
  font-size: 16px;
}
.container {
  width: 80%;
  padding: 2rem;
  min-height: 50vh;
}
h1 {
  font-size: 2rem;
}
p {
  font-size: 1em;
}
```



© Copyright Intellipaat. All rights reserved.

Hands-On 5 — Build a Responsive Content Section Using CSS Units

Create a content section containing:

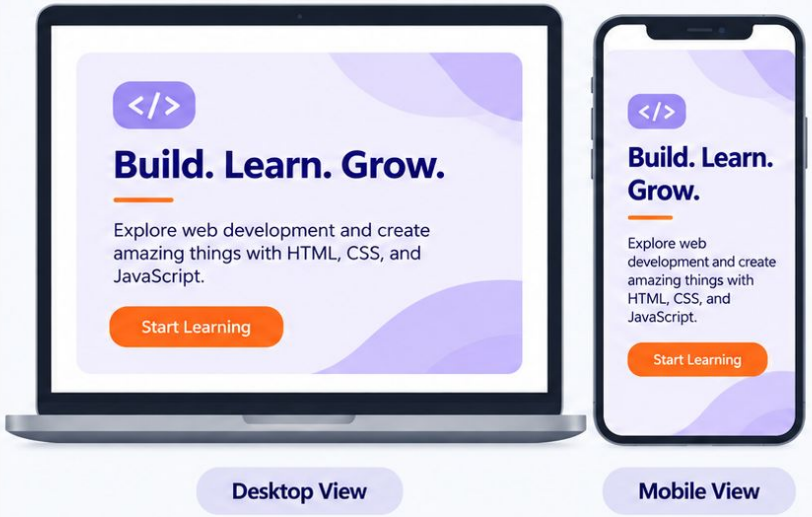
- A heading
- A paragraph
- A container for the content

Your Task

Use the CSS units covered in the module:

1. Use **px** for a fixed measurement.
2. Use **%** for a value relative to the parent/container.
3. Use **rem** for typography or spacing relative to the root font size.
4. Use **em** for a value relative to the relevant font size.
5. Use **vh** for the section's height.
6. Use **vw** for a width-related value.

Visual Preview (Responsive Design)



Expected Result: A section that demonstrates how different CSS units produce different sizing relationships.

© Copyright Intellipaat. All rights reserved.

Specificity, Cascade & Inheritance

© Copyright Intellipaat. All rights reserved.

CSS Specificity

CSS specificity is the mechanism the browser uses to determine **which CSS rule should take priority when multiple rules target the same element**.

Example Code

HTML

```html
<p class="text" id="message">Hello</p>
```

CSS

```css
p {
  color: blue;
}

.text {
  color: green;
}

#message {
  color: red;
}
```

Output

Hello

Specificity Comparison

Selector	Type	Specificity	Color
p	Element	0, 0, 0, 1	blue
.text	Class	0, 0, 1, 0	green
#message	ID	0, 1, 0, 0	red

Explanation

1. The element selector `p` sets the text color to blue.
2. The class selector `.text` sets the text color to green.
3. The ID selector `#message` sets the text color to red.
4. All three selectors target the same paragraph, but the paragraph will be **red** because the ID selector has **higher specificity** than the class and element selectors.

© Copyright Intellipaat. All rights reserved.

CSS Specificity

A **webpage** can have many CSS rules, and several of them may apply to the same element.

The browser therefore needs a way to decide:

Which rule should win?

Specificity provides that priority.

Basic Specificity Hierarchy

For the selectors covered in this module, the basic order is:

Element < Class < ID < Inline style

© Copyright Intellipaat. All rights reserved.

CSS Specificity

Specificity is determined by **what selector is used to target an element**, not by where the CSS file happens to be located. For example, an ID selector in an external stylesheet can be more specific than an element selector written in an internal stylesheet.

When Specificity Becomes Important

Specificity matters when:

1. Multiple rules target the same element
2. Those rules set the same property
3. The browser needs to determine which declaration should be used

If two rules don't conflict — for example, one sets the `color` and another sets `font-size` — specificity doesn't prevent both from being applied.

Example

```
p {
  color: blue;   /* sets color */
}

.text {
  font-size: 20px; /* sets font size */
}
```

Both rules are applied

because they set different properties.

Important distinction

Specificity is **not** **simply "the last rule wins."**

The browser considers the priority of the selectors first.

The position of a rule in the CSS can matter when competing declarations have **equal specificity**.

Example

```
/* In style.css (external) */
#message {
  color: red;
}

/* In a <style> tag (internal) */
p {
  color: blue;
}
```

The text will be **red** because the **ID** selector (`#message`) has **higher specificity** than the element selector (p), even though it is in an external stylesheet.

The detailed interaction between specificity and the **cascade** will be covered in the following topics.

© Copyright Intellipaat. All rights reserved.

CSS Cascade

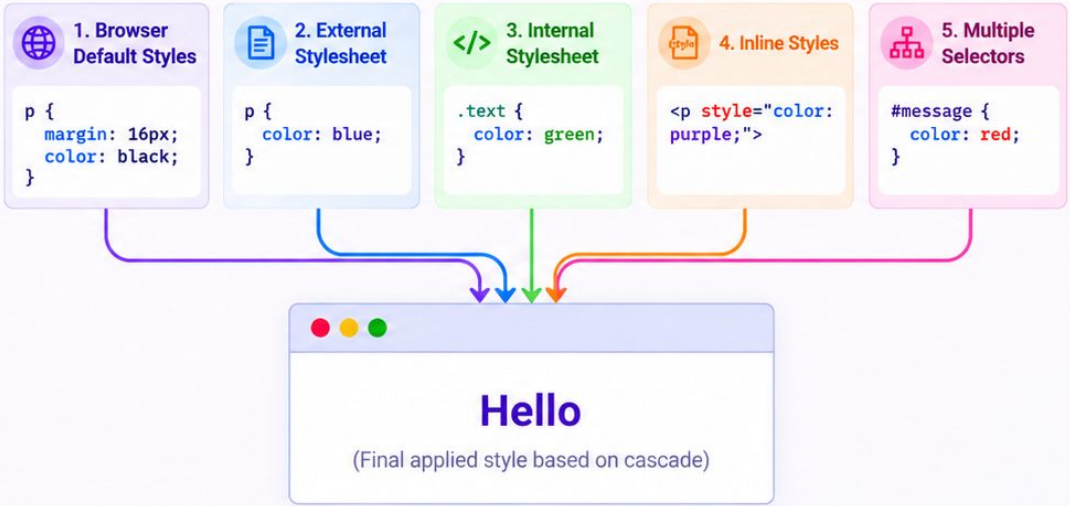
The **cascade** is the process CSS uses to determine **which declarations are ultimately applied when multiple styles could affect the same element.**

A single HTML element can be affected by CSS from **different places and by multiple rules.**

For example, an element might receive styles from:

1. A browser's default styles
2. An external stylesheet
3. An internal stylesheet
4. Inline styles
5. Multiple selectors within those stylesheets

The cascade determines **which declarations should take effect** when these styles interact.



The cascade considers multiple factors (like origin, specificity, and source order) to decide the **final style.**

© Copyright Intellipaat. All rights reserved.

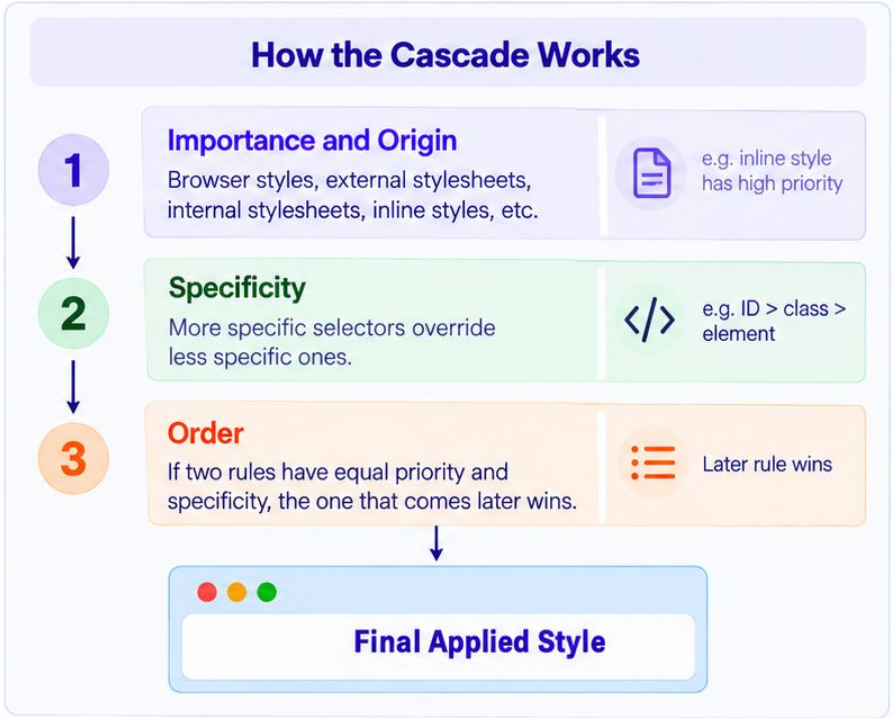
CSS Cascade

Specificity is one part of the **cascade**, but the cascade is the broader process used to resolve competing CSS declarations.

At an introductory level, students should understand three important ideas:

1. **Importance and origin** — Where the rule comes from and whether it has special priority.
2. **Specificity** — How specifically the selector targets the element.
3. **Order** — If competing rules have equal priority and specificity, the later one can win.

You don't need to go deeply into the full CSS specification here.



© Copyright Intellipaat. All rights reserved.

Example

1 Same Selector, Same Specificity

CSS

```css
p {
  color: blue;
}

p {
  color: red;
}
```

Both rules target the same elements and have the same specificity.

Because the **second declaration comes later**, the final color is:

red

Output

This is a paragraph.

2 Different Selectors, Different Specificity

CSS

```css
p {
  color: blue;
}

.text {
  color: red;
}
```

The `.text` rule has **higher specificity** than the `p` rule.

Therefore, the `.text` rule takes priority over the `p` rule.

Output

This is a paragraph.

(with class="text")

© Copyright Intellipaat. All rights reserved.

Cascade vs. Specificity

These two concepts are related, but they are **not the same**.

Specificity

Specificity answers:

"Which selector is more specific?"

Determines how strongly a selector targets an element.

Cascade

The cascade answers:

"Which declaration should ultimately be applied?"

The overall process used to determine which competing declarations are applied.

Key Differences

Aspect	Specificity	Cascade
Quection it answers	Which selector is more specific?	Which declaration should ultimately be applied?
What it does	Measures the strength of a selector	Resolves all conflicts using multiple factors
Scope	Only one factor	Overall process (uses origin, specificity, order, etc.)

Specificity is **one of the factors** used by the cascade to resolve conflicts.

© Copyright Intellipaat. All rights reserved.

CSS Inheritance

CSS Inheritance is the mechanism through which certain CSS property values are **passed from a parent element to its child elements**.

How Inheritance Works

Consider:

HTML

```
<div>
   <p>Hello</p>
   <p>Welcome</p>
</div>
```

If we apply:

CSS

```
div {
   color: blue;
}
```

Output

Hello

Welcome

The `<p>` elements can inherit the `color` value from their parent `<div>`, so their text appears **blue**.

The CSS does not need to explicitly set `color: blue` on every paragraph.

© Copyright Intellipaat. All rights reserved.

CSS Inheritance

Parent → Child

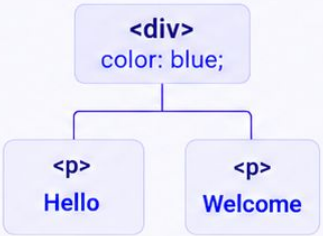
The basic idea is:

Parent element has an inherited property → Child receives that value

For example:

HTML

```
<div style="color: blue;">
    <p>Hello</p>
    <p>Welcome</p>
</div>
```



This can continue through multiple levels of nested elements.

Not All Properties Inherit

An important point is that **inheritance does not apply to every CSS property.**

Properties that normally inherit

Some properties normally inherit, such as:

- `color`
- `font-family`
- `font-size`

Properties that normally do not inherit

Other properties normally do not inherit, such as:

- `margin`
- `padding`
- `border`
- `width`
- `height`

© Copyright Intellipaat. All rights reserved.

Inherited vs. Directly Applied Styles

If a **child** has its **own value** for an inherited property, that value takes **precedence** over the inherited value.

CSS

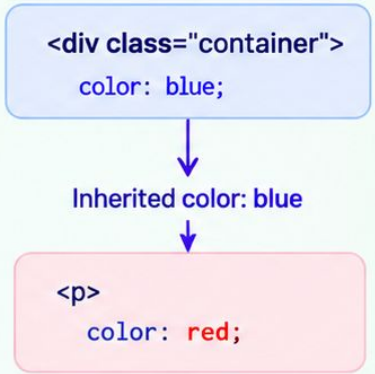
```css
.container {
    color: blue;
}

p {
    color: red;
}
```

HTML

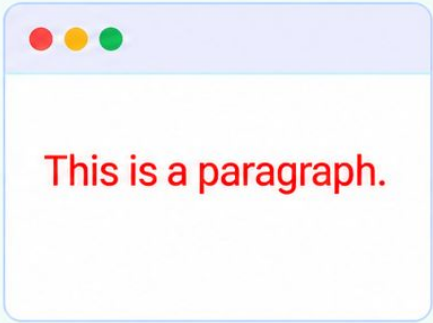
```html
<div class="container">
    <p>This is a paragraph.</p>
</div>
```

How It Works



The <p> element has its own specified value, so it overrides the inherited value from the parent.

Output



© Copyright Intellipaat. All rights reserved.

Hands-On 6 — Resolve Conflicting CSS Styles

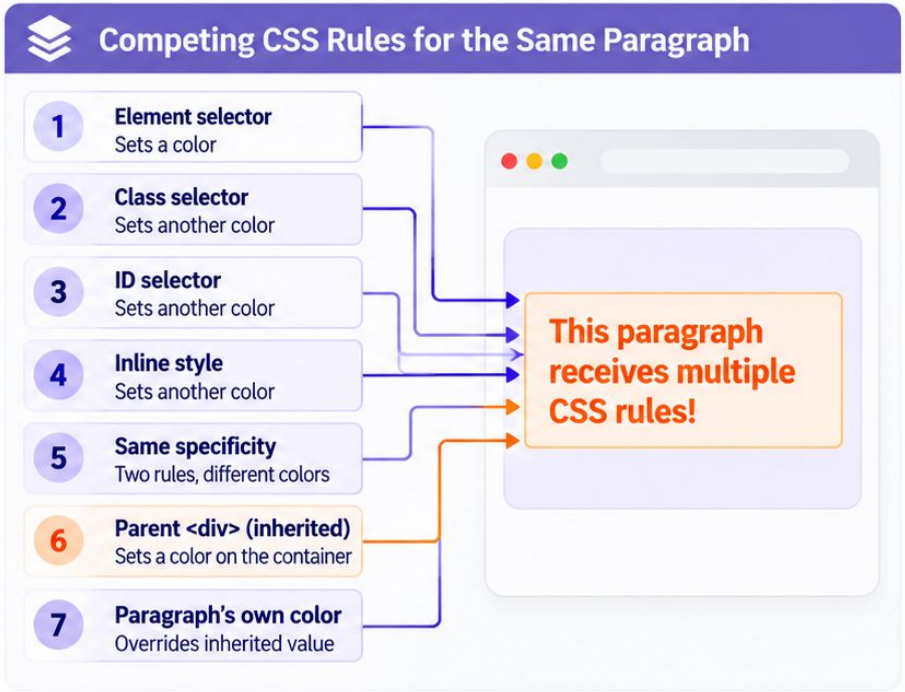
Create the following HTML structure:

- A `<div>` container
- A paragraph inside the container

Apply multiple CSS rules that target the same paragraph.

Your Task

1. Apply a color to the paragraph using an **element selector**.
2. Apply another color using a **class selector**.
3. Apply another color using an **ID selector**.
4. Add an **inline style** with another color.
5. Create two rules with the **same selector and specificity**, but different colors.
6. Set a color on the parent `<div>` and observe whether the paragraph inherits it.
7. Give the paragraph its own color and observe what happens to the inherited value.



Expected Result: Students can explain why a particular color wins and distinguish between an **inherited value** and a **directly applied value**.

© Copyright Intellipaat. All rights reserved.

Thanks!



080-4524-9465



support@intellipaat.com

© Copyright Intellipaat. All rights reserved.